ESOF-3675

Movie Recommendation System.

Abdul Khan

Nidhi Patel

27th-Mar-2025

Contents

Declaration *4*

Contribution **4**

1 Introduction *5*

1.1 Overview **5**

1.2 Problem Definition and Motivation **5**

1.2.1 Problem: 5

1.2.2 Motivation: 5

1.3 Objectives and Scope **6**

1.3.1 Primary Objective: 6

1.3.2 Scope: 6

1.4 Technical Challenges **6**

1.5 Research Approach and Hypothesis **6**

2 Related work/Literature Review *7*

3. Methodology *8*

4 Data Pre-processing *8*

4.1 Data Summary **8**

4.2 Encoding Categorical Variables **9**

4.2.1 Ordinal Encoding: Content Rating 9

4.2.1.1 Encoding Breakdown 9

4.2.2 Nominal Encoding: Color Column 9

4.3 Scaling **11**

4.4 Handling Missing Values **11**

4.4.1 Dropping Rows with Critical Missing Values 13

4.4.2 Imputing Missing Values for Numerical Columns 13

4.4.3 Handling Missing Values for Categorical Columns 13

4.4.4 Ensuring Data Integrity in Scaled Features 13

4.5 Duplicate Removal and Data Standardization **13**

4.5.1 Duplicate Detection and Removal 14

4.5.2 Data Formatting and Standardization 14

5 Data Visualization *14*

5.1 Histograms: **14**

5.1.1 IMDb Score Distribution (Figure 1) 14

5.1.2 Gross Earnings Distribution (Figure 2) 15

5.1.3 Budget Distribution (Figure 3) 16

5.1.4 Number of User Reviews (Figure 4) 17

5.1.5 Number of Critic Reviews (Figure 5) 17

5.1.6 Movie Duration Distribution (Figure 6) 18

5.2 Outlier Detection **19**

5.2.1 Identified Outliers 19

5.2.2 Approaches for Handling Outliers 22
5.3 Heatmap: 22
5.4 Relevance 24
6 Database Design 24
6.1 ER Diagram (Figure 14): 25
6.2 Entity Relationships 25
6.3 Constraints 25
7 Database Implementation 25
7.2 Data Storage in MongoDB 25
7.3 Indexing Implementation 26
7.3.1 Indexes Implemented: 26
7.3.3 Performance Analysis 26
7.4 Genre Transformation 27
7.4.1 Splitting the Genres Column into a List 27
7.4.2 One-Hot Encoding with MultiLabelBinarizer 27
7.5 Data Embedding for Optimization 27
7.5.1 Structure of the Embedded Model 28
7.5.2 Advantages of Full Embedding 28
7.5.3 Limitations and Trade-offs 28
7.5.4 Conclusion 28
8 Data Mining 29
8.1 Association Rule Mining for Genre Analysis 29
8.1.1 Overview: 29
8.1.2 Methodology: 29
8.1.3 Output: 29
8.1.4 Interpretation: 30
8.1.5 Conclusion: 30
8.3 Gradient Boosting for Predictive Modeling 31
8.3.1 Objective and Rationale: 31
8.3.2 Input Features and Preprocessing: 31
8.3.3 Model Configuration: 31
8.3.4 Performance Metrics: 31
8.3.5 Feature Importance Analysis: 31
8.3.6 Interpretation 31
8.3.7 Overall Conclusion from Predictive Modeling: 32
8.4 Scatter-plot analysis 33
8.4.1. IMDb Score vs. Gross 33
8.4.2. IMDb Score vs. Budget 33
8.4.3. IMDb Score vs. Critic Reviews 34
8.4.4 Conclusion: Focusing on IMDb Score and Genre for Recommendations 34
8.5 Clustering with K-Means 35
8.5.1 Overview 35
8.5.2 Clustering Methodology 35

8.5.3 Output Statistics 35
8.5.4 Implications for Recommendations 35

8.6 Cosine Similarity for Recommendation Ranking 36
9.6.1 Overview: 36
8.6.2 Methodology: 36
8.6.3 Output Statistics: 36
8.6.4 Implications: 36
8.6.5 Conclusion: 37

9. Graphical User Interface (GUI) and System Integration 37

9.1 Overview 37

9.2 File Structure and Organization 37
9.2.1 app.py 37
9.2.2 templates/ 38
9.2.3 static/ 38
9.2.4 users.json 38

9.3 GUI Pages Description 39
9.3.1 Login Page (login.html) 39
9.3.2 Signup Page (signup.html) 39
9.3.3 Results Page (results.html) 40
9.3.4 Recommendations Page (recommendations.html) 40

9.4 Backend Integration 40
9.4.1 Routing and Application Logic 41
9.4.2 Python Logic and Business Rules 41
9.4.3 User Authentication and Security 41
9.4.2 Dynamic Page Rendering with Jinja2 42

9.5 User Authentication and Security 42
9.5.1 Password Hashing with Flask-Bcrypt 42
9.5.2 Session Management with Flask-Login 43
9.5.3 User Storage in users.json 43

9.6 Movie Posters Integration 44

9.7 Deployment with GitHub and Render 44

10. Conclusion 45

11. Appendix 46

Appendix A: GitHub Repository for Source Code 46

Appendix B: Live Web Application 47

Appendix C: Dataset Source – IMDb 5000 Movie Dataset 47

Appendix D: Cosine Similarity Function (Python) 47

Appendix E: Sample MongoDB Document Structure 47

Appendix F: Gradient Boosting Model Configuration 48

Appendix G: Required Python Libraries and Tools 48

Declaration

I hereby declare that this **Movie Recommendation System** is entirely my own work except for the references quoted and it has not received any previous academic credit at this or any other institution. I declare that I have read the University's Academic Integrity and Policies on plagiarism and copyright protection and that I have upheld them in producing this work. The content of this report does not contain any part of the previously submitted report(s) by any means.

Contribution

Student Name	Algorithm Development and Coding Contribution (%)	Writing and Illustration Contribution (%)
Nidhi Patel	50%	60%
Abdul Khan	60%	40%

Nidhi Patel
1179002
27th-Mar-25

Abdul Khan
1112317
27th-Mar-25

1 Introduction

1.1 Overview

This project initially started with a COVID-19 dataset. We began cleaning the data but encountered challenges in making predictions. Since it was too early to predict trends due to a lack of data on how COVID-19 vaccination would affect different age groups and other factors, we decided to shift our focus to a movie recommendation system. This system focuses on developing a content-based movie recommendation system that leverages a rich, preprocessed movie dataset. The dataset includes a variety of attributes such as movie titles, genres, IMDb scores, budgets, critic reviews, and more. Each movie is stored as a JSON-like document in MongoDB Atlas—a fully managed cloud-hosted NoSQL database. MongoDB Atlas provides the flexibility to handle semi-structured data and ensures scalability and high availability, which is critical for supporting real-time recommendation queries.

1.2 Problem Definition and Motivation

1.2.1 Problem:

The enormous volume of movie data available online makes it challenging for users to discover films that truly match their interests. Many existing recommendation systems rely on collaborative filtering, which can struggle with the cold-start problem (i.e., new or less popular movies) and may not capture the nuanced content characteristics that define a user's taste.

1.2.2 Motivation:

Our goal is to create a recommendation system that primarily relies on content-based features, such as IMDb score and genre information, to generate personalized suggestions. By focusing on these features, the system can:

- Offer relevant recommendations even for niche or newly released movies.
- Overcome the limitations of collaborative filtering by emphasizing intrinsic movie attributes.
- Enhance user satisfaction through tailored suggestions based on quality (IMDb score) and thematic similarity (genres).

1.3 Objectives and Scope

1.3.1 Primary Objective:

Develop a scalable, content-based movie recommendation system that uses IMDb scores and genre information as the primary drivers for recommendations.

1.3.2 Scope:

- **Data Preprocessing:** Handle missing values appropriately, Encode categorical and standardize numerical features to ensure uniform scaling across different attributes.
- **Database Design:** Utilize MongoDB Atlas to store the preprocessed movie data. Implement an embedded document and Apply indexing strategies on key fields (e.g., movie title, director, genres, IMDb score) to enhance query performance.
- **Data Implementation:** Load and transform the raw movie dataset into a clean, structured format. Store the transformed data in the MongoDB Atlas environment.
- **Data Mining:** Use gradient boosting to predict the log-transformed gross revenue based on features such as budget, critic reviews, and IMDb score. Apply K-Means clustering to group similar movies and compute cosine similarity within clusters to rank movies and provide personalized recommendations.
- **Graphical User Interface (GUI):** Develop an intuitive user interface to present recommendations. Enable users to input their movie of choice and receive a tailored list of similar movies.

1.4 Technical Challenges

- **Data Quality:** The movie dataset is heterogeneous, with missing values and inconsistencies that must be carefully handled.
- **Feature Transformation:** Converting multi-label genre strings into a machine-readable format through one-hot encoding and encoding ordinal variables.
- **Data Storage and Query Efficiency:** Implementing a robust, embedded data model in MongoDB Atlas to support rapid query execution, alongside strategic indexing to optimize search performance.
- **Model Integration:** Leveraging advanced machine learning techniques such as gradient boosting, K-Means clustering, and cosine similarity to both validate feature importance and generate personalized recommendations.

1.5 Research Approach and Hypothesis

Our hypothesis is that a content-based recommendation system relying on intrinsic movie attributes—specifically, IMDb score and genre—can deliver high-quality, personalized recommendations. We approach this problem by:

- Preprocessing and transforming the raw movie data.

- Validating the importance of key features using predictive models.
- Using clustering techniques to group movies and refine recommendations through similarity measures. The anticipated outcome is a system that not only recommends movies effectively but also demonstrates that the chosen features are robust predictors of movie quality and popularity.

2 Related work/Literature Review

The field of recommendation systems has evolved significantly over the past two decades, driven by the increasing need to personalize content in digital platforms such as Netflix, Amazon, and Spotify. These systems are typically categorized into three types: **content-based filtering**, **collaborative filtering**, and **hybrid approaches**.

Content-based filtering relies on the characteristics of items and user preferences. For instance, if a user enjoys action movies, the system recommends other action movies based on genre, director, or IMDb score. This approach is especially effective in solving the *cold-start problem*—when a new movie lacks user interaction data. Our project leverages this approach by using intrinsic attributes like IMDb scores and genres to provide recommendations.

Collaborative filtering, on the other hand, uses the behavior of similar users to generate suggestions. While effective in some scenarios, it suffers from **data sparsity** and **cold-start issues**, especially when user interaction data is limited. **Hybrid systems** attempt to combine both methods to overcome individual weaknesses, as demonstrated by Netflix's early recommendation algorithm, which mixed genre-based filtering with user ratings.

Several research studies support the effectiveness of content-based filtering. *Lops et al. (2011)* outlined how machine learning and metadata could power content-aware recommendations, while *Adomavicius and Tuzhilin (2005)* emphasized the importance of context and personalization. Moreover, systems like IMDb and TMDb use metadata such as cast, crew, and keywords to enhance recommendations.

Our system integrates principles from these established approaches while incorporating additional techniques such as **clustering** and **cosine similarity**. This ensures that recommendations are not only accurate but also diverse and relevant to user interests. By focusing on content rather than user interaction history, our model avoids the pitfalls of collaborative systems and maintains high relevance even for less popular or new movies.

3. Methodology

The methodology begins by **preprocessing** the raw dataset, handling missing values through imputation and encoding categorical variables to ensure consistency. Next, a **database design** step conceptualizes an entity-relationship model, though the final **data implementation** in MongoDB Atlas uses a fully embedded document structure with strategic indexing for efficient querying. The **data mining** phase applies predictive modeling (Random Forest and Gradient Boosting) to validate key features, clusters movies with K-Means, and computes cosine similarity within clusters to generate personalized recommendations; association rule mining further uncovers frequent genre co-occurrences. Finally, a **GUI** presents these recommendations in a user-friendly manner, consolidating the entire workflow into a cohesive and interactive system.

4 Data Pre-processing

4.1 Data Summary

The dataset consists of 5043 rows and 26 columns. The data types are as follows: float64 (12 columns), int64 (2 columns), and object (12 columns).

Statistic	duration_original	num_critic_for_reviews_original	gross_original	num_user_for_reviews_original	budget_original	title_year	imdb_score
count	5028.000000	4993.000000	4150.000000	5022.000000	4551.000000	4922.000000	5028.000000
mean	107.201470	140.507109	4.846960e+07	107.521289	3.551028e+07	2002.473517	6.442138
std	25.197441	121.601675	6.452998e+07	186.873405	4.320703e+07	14.077084	1.125116
min	7.000000	1.000000	1.625390e+06	1.000000	2.180000e+02	1916.000000	1.600000
25%	93.000000	50.000000	1.197000e+06	11.000000	8.000000e+06	1999.000000	5.700000
50%	104.000000	110.000000	2.929500e+06	40.000000	2.000000e+07	2006.000000	6.600000
75%	118.000000	194.000000	6.425000e+06	103.000000	4.000000e+07	2016.000000	7.400000
max	511.000000	642.000000	7.680580e+08	5060.000000	1.221549e+09	2035.000000	9.500000

Table 1: Data Statistics

Table 1: The data statistics above provide insights into the dataset's numerical attributes, including their distributions and statistical properties. A statistical summary of the dataset was generated, listing all data types and key statistical measures such as mean, minimum, maximum, and standard deviation.

4.2 Encoding Categorical Variables

Since our dataset has some text-based (categorical) features, we needed to convert them into numbers so they could be used in analysis. We used two types of encoding: **ordinal encoding** and **nominal encoding**.

4.2.1 Ordinal Encoding: Content Rating

The **content rating** (like PG, PG-13, R, etc.) was turned into numbers from **0 to 18(Table 2)**. The **content rating** of a movie determines its suitability for different audiences based on age restrictions and guidelines. Since these ratings follow a **natural order** (from least to most restrictive), we assigned numerical values to represent them in a structured way.

4.2.1.1 Encoding Breakdown

- **0-2:** Undefined or Unrated (e.g., Unknown, Unrated, Not Rated)
- **3-5:** General Audience (e.g., Approved, Passed, G)
- **6-8:** Parental Guidance (e.g., PG, PG-13, R)
- **9-12:** Stronger Restrictions (e.g., NC-17, X, GP, M)
- **13-18:** Television-Specific Ratings (e.g., TV-G, TV-Y, TV-PG, TV-14, TV-MA)

By using ordinal encoding, we kept the ranking structure, where higher numbers indicate more age restrictions.

4.2.2 Nominal Encoding: Color Column

The **color column** tells us whether a movie is in **color or black-and-white**. Since this feature doesn't have a natural order (one isn't "greater" than the other), we used **nominal encoding (Table 2)**, where each category was assigned a unique number:
• **Black & White** → 0
• **Color** → 1
• **Unknown** (if the color info was missing) → 2
This method lets us keep all the information while making sure the model can understand it.
Refer to table 2 for encoded values.

Content Rating	Encoded Rating	Color	Encoded Color

Unknown	0	Color	1
Unknown	0	Black and White	0
Unknown	0	Unknown	2
Unrated	1	Black and White	0
Unrated	1	Color	1
Not Rated	2	Color	1
Not Rated	2	Black and White	0
Approved	3	Color	1
Approved	3	Black and White	0
Passed	4	Black and White	0
Passed	4	Color	1
G	5	Color	1
G	5	Black and White	0
PG	6	Unknown	2
PG	6	Color	1
PG	6	Black and White	0
PG-13	7	Color	1
PG-13	7	Unknown	2
PG-13	7	Black and White	0
R	8	Color	1
R	8	Unknown	2
R	8	Black and White	0
NC-17	9	Color	1
NC-17	9	Black and White	0
X	10	Black and White	0
X	10	Color	1
GP	11	Color	1
M	12	Color	1
TV-G	13	Color	1
TV-G	13	Black and White	0
TV-Y	14	Color	1
TV-Y7	15	Color	1
TV-PG	16	Color	1
TV-14	17	Color	1
TV-MA	18	Unknown	2
TV-MA	18	Color	1

Table 2: Encoded Values

4.3 Scaling

To make sure all the numerical features in our dataset are on the same level, we standardized them. This means we changed the values so that they have an **average (mean) of 0** and a **standard deviation of 1**. Doing this helps because some features might have big numbers while others have small ones, and we don't want any feature to have too much influence just because of its scale. We **did not** standardize **tittle year** and **IMDb score** because, **title year** does not have a very long range and keeping it in its original form makes more sense. **IMDb score** is already a rating from 1 to 10, so changing it would make it harder to understand. By standardizing the other numerical features, we made sure they are fair and balanced, so no single feature has too much influence when we analyze the data or build a model.

4.4 Handling Missing Values

Handling missing values was a crucial step in preparing the dataset for further analysis and storage. Given the diverse nature of the attributes, different strategies were implemented based on the type of data to ensure data integrity and minimize potential biases.

Column	**Missing Values**
movie_title	0
genres	1
duration_original	15
director_name	104
color	20
num_critic_for_reviews_original	50
actor_2_name	13
gross_original	884
actor_1_name	7
actor_3_name	23
plot_keywords	153
movie_imdb_link	0
num_user_for_reviews_original	21
language	14
country	5
content_rating	80
budget_original	492
title_year	108
imdb_score	0
duration_scaled	15

num_critic_for_reviews_scaled	50
num_user_for_reviews_scaled	21
gross_scaled	884
budget_scaled	492
content_rating_encoded	80
color_encoded	20

Table 2: No. of Missing Values

Column	**Value**
movie_title	0.997442
genres	0.297049
duration_original	0.201842
director_name	2.602605
color	0.001403
actor_2_name	0.257203
gross_original	17.582846
actor_1_name	0.193408
actor_3_name	0.046197
plot_keywords	0.000000
movie_imdb_link	0.436319
num_user_for_reviews_original	0.111903
language	0.277703
country	0.001194
content_rating	0.766598
budget_original	9.750492
title_year	0.001552
imdb_score	0.141582
duration_scaled	0.697148
num_critic_for_reviews_scaled	0.491473
num_user_for_reviews_scaled	0.396409
gross_scaled	17.529480
budget_scaled	9.756098
content_rating_encoded	0.750000
color_encoded	0.002

Table 3:Percentage of Missing Values

The number (Table 3) and percentage (Table 4) of missing values for each column have been calculated.

4.4.1 Dropping Rows with Critical Missing Values

Certain attributes, such as 'movie_title', are fundamental to the dataset, and any missing values in these fields would render a record unusable. Although we had no missing values for this column.

4.4.2 Imputing Missing Values for Numerical Columns

For numerical columns, missing values were handled differently based on the nature of the data:

- Median Imputation:
  Columns such as 'budget_original', 'gross_original', 'num_critic_for_reviews_original', and 'num_user_for_reviews_original' had missing values imputed using the median of the respective column. This approach prevents extreme values from skewing the data compared to mean imputation.
- Indicator Value for Missing Data:
  Some numerical attributes, including 'title_year', and 'duration_original', had missing values replaced with -1 to indicate the absence of data. This approach ensures that missing values do not interfere with computations while preserving the distinction between known and unknown values.

4.4.3 Handling Missing Values for Categorical Columns

Categorical features such as 'color', 'actor_1_name', 'actor_2_name', 'actor_3_name', 'director_name', 'content_rating', 'language', 'country', 'genres', and 'plot_keywords' had missing values replaced with "Unknown" to ensure consistency in data representation. This approach allows categorical variables to retain a structured format without losing valuable records due to missing values.

4.4.4 Ensuring Data Integrity in Scaled Features

Scaled versions of numerical columns ('duration_scaled', 'num_critic_for_reviews_scaled', 'num_user_for_reviews_scaled', 'gross_scaled', 'budget_scaled') required careful handling. Initially, missing values were incorrectly assigned "Unknown", which was later corrected by ensuring that missing values in these scaled columns were replaced with a designated numeric indicator (-10) to prevent errors in future analyses.

4.5 Duplicate Removal and Data Standardization

After handling missing values, duplicate records were identified and removed to eliminate redundancy and ensure data integrity. This step was essential in preventing duplicate entries from skewing analyses and misleading insights.

4.5.1 Duplicate Detection and Removal

Duplicates were identified based on key attributes, including 'movie_title', 'director_name', and 'title_year'. If multiple records contained the same values for these fields, only the first occurrence was retained while duplicates were removed. This ensured that each movie entry remained unique within the dataset.

4.5.2 Data Formatting and Standardization

To maintain consistency across the dataset, several formatting transformations were applied:

- Text Standardization:
  - Movie titles, director names, and actor names were trimmed of extra spaces and capitalized correctly to ensure uniformity in textual data.
  - Content rating values were converted to uppercase for standardization.
- Numerical Type Conversion:
  - Fields such as 'budget_original', 'gross_original', and 'title_year' were converted to integer types instead of floating-point values. This optimization reduced memory usage and improved computational efficiency in later stages.

By implementing these **duplicate removal and standardization** steps, the dataset was made more structured, ensuring that stored data remained unique, correctly formatted, and optimized for further processing.

5 Data Visualization

Histograms, box plots, and QQ plots were generated for numerical attributes. Correlation heatmap to visualize relationships between features. Proper labels and titles added to all plots.

5.1 Histograms:

5.1.1 IMDb Score Distribution (Figure 1)

- The IMDb scores follow a nearly normal distribution, peaking around **6-7**, indicating that most movies have moderate ratings.
- Very few movies have extremely high or low scores, making extreme ratings less common.
- This distribution helps in categorizing movies into **high-rated, moderate, and low-rated**, allowing users to filter based on their preferences.

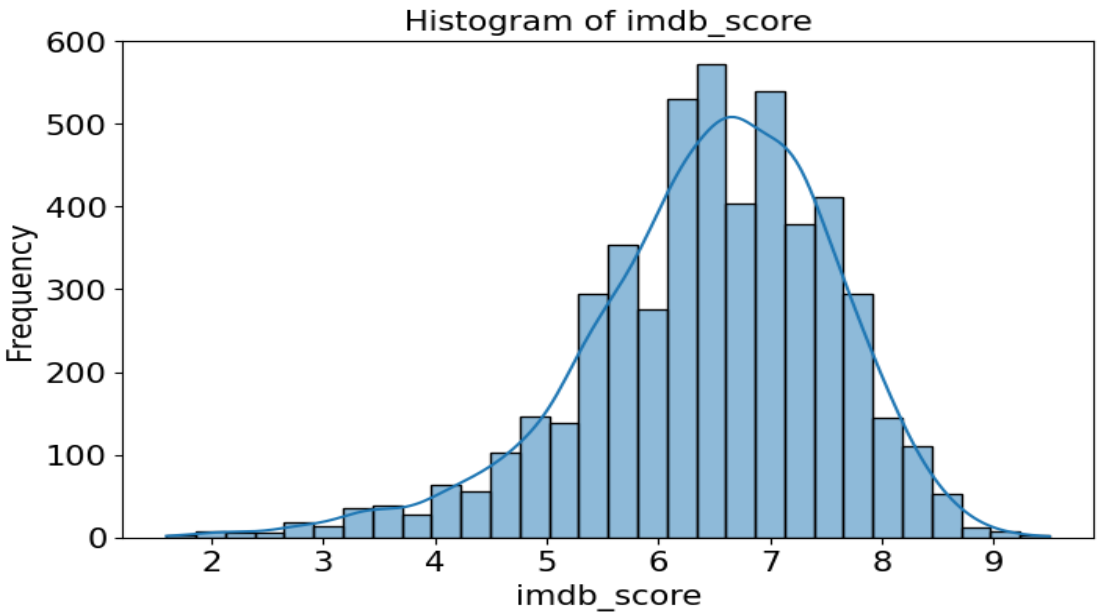

Figure 1: Histogram of imdb_score

5.1.2 Gross Earnings Distribution (Figure 2)

- The distribution is **highly right-skewed**, meaning most movies earn significantly less, while a small number achieve blockbuster-level earnings.
- The presence of extreme outliers suggests that only a handful of movies generate exceptionally high revenue.
- This data is useful for **financial analysis**, profitability assessments, and for users interested in highly successful movies.

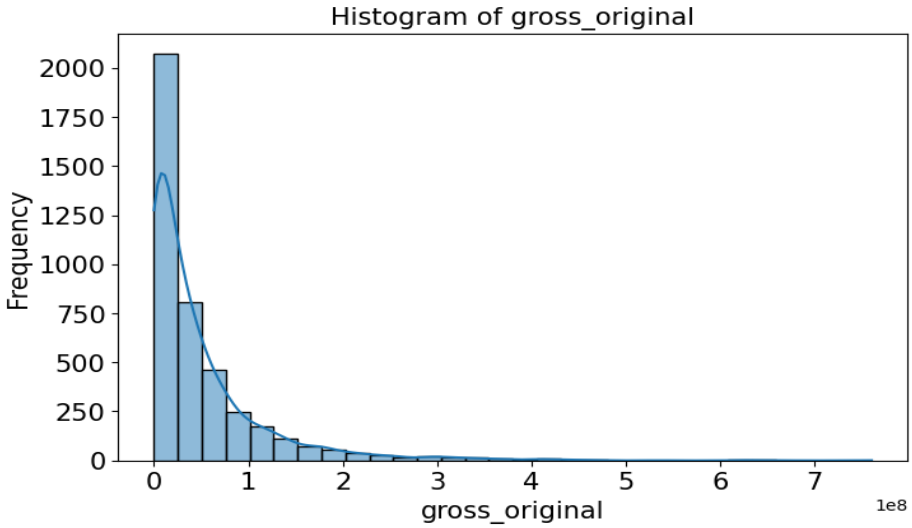

Figure 2: Histogram of gross_original

5.1.3 Budget Distribution (Figure 3)

- The budget data is **skewed right**, indicating that most movies have smaller budgets, with only a few surpassing **$1 billion**.
- This segmentation helps classify movies into **indie, mid-range, and big-budget categories**, which could be useful for users who prefer certain production scales.

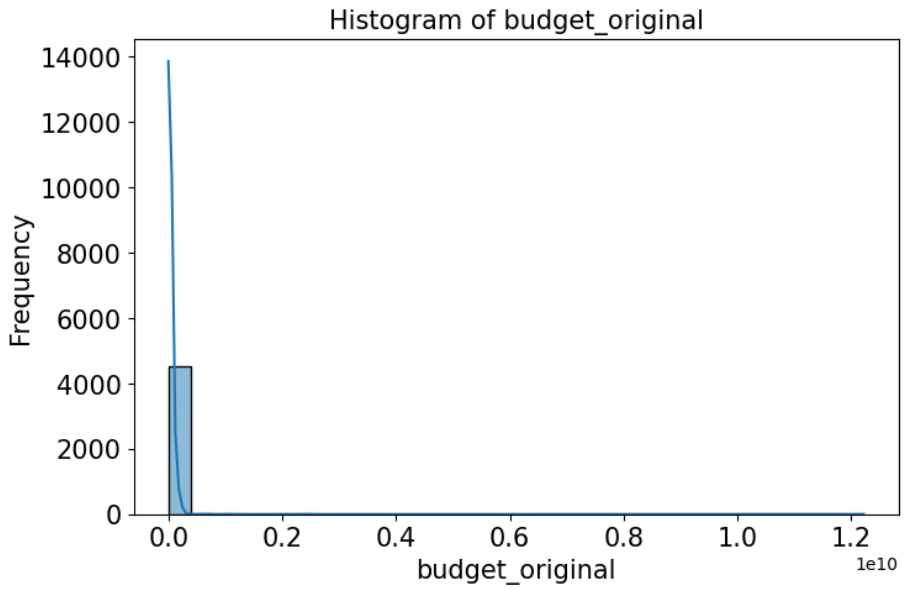

Figure 3: Histogram of budget_original

5.1.4 Number of User Reviews (Figure 4)

- Most movies receive **a small number of user reviews**, while a few popular ones have thousands.
- This metric is essential for identifying **trending and widely discussed movies** that users might be interested in.
- It also helps in setting **review-based ranking criteria** for recommendations.

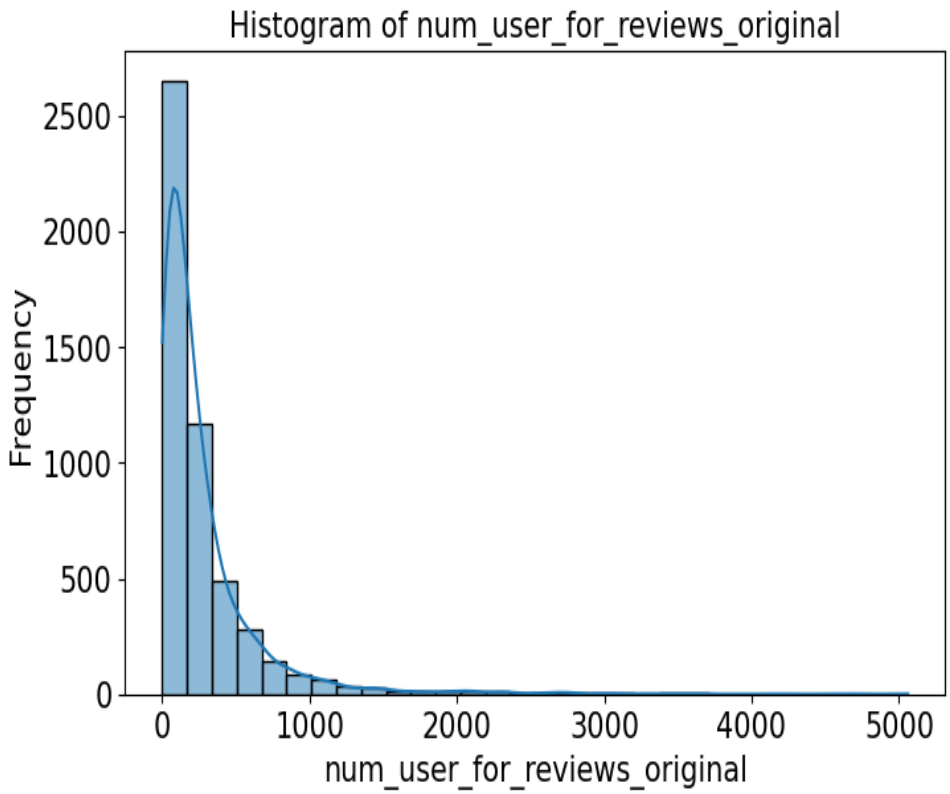

Figure 4: Histogram of num_user_for_reviews_original

5.1.5 Number of Critic Reviews (Figure 5)

- Similar to user reviews, critic reviews follow a **right-skewed distribution**, with fewer movies receiving a high number of critiques.
- It helps distinguish between **audience favorites and critic favorites**, which can be a useful filtering criterion for users who prefer critically acclaimed films.

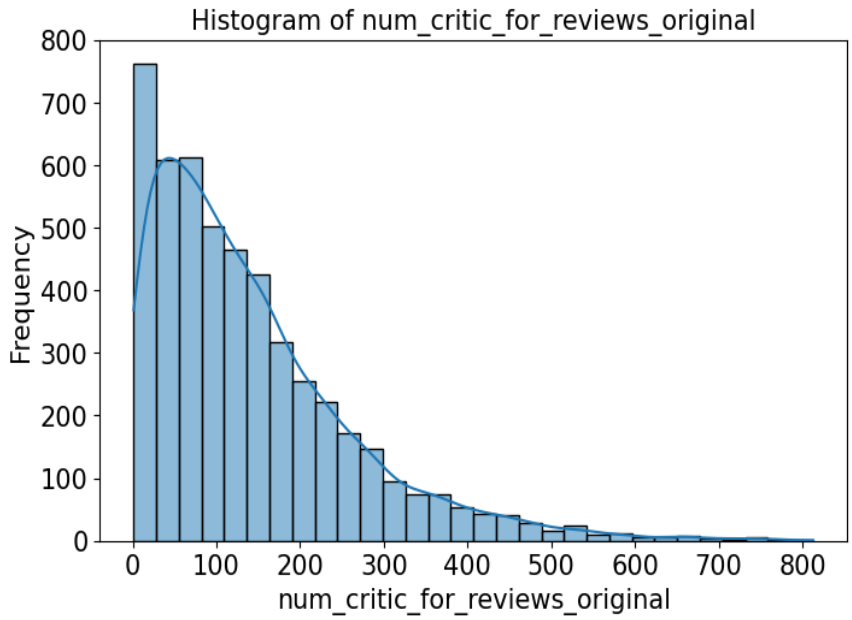

Figure 5: Histogram of num_critic_for_reviews_original

5.1.6 Movie Duration Distribution (Figure 6)

- The runtime of movies is **normally distributed**, with most movies between **90-150 minutes**.
- Very few films exceed **200 minutes**, making them outliers.
- This helps in filtering movies by **short, medium, and long runtime**, catering to user preferences based on available watch time.

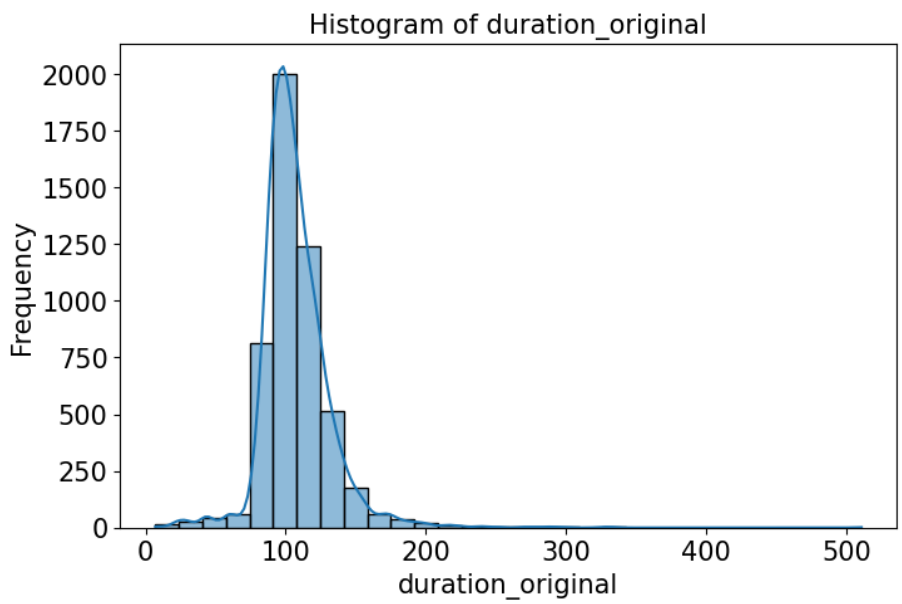

Figure 6: Histogram of duration_original

5.2 Outlier Detection

The box plots reveal several significant outliers across different movie attributes. These outliers can distort analysis and impact the effectiveness of a recommendation system if not handled properly. Rather than removing them outright, a strategic approach is required to maintain data integrity while minimizing their influence on recommendations.

5.2.1 Identified Outliers

5.2.1.1 IMDb Score (Figure 7):

- A few extreme values on both ends, with some movies rated exceptionally low (<3) or very high (>9).
- These could be films with strong cult followings or universally loved/hated movies.

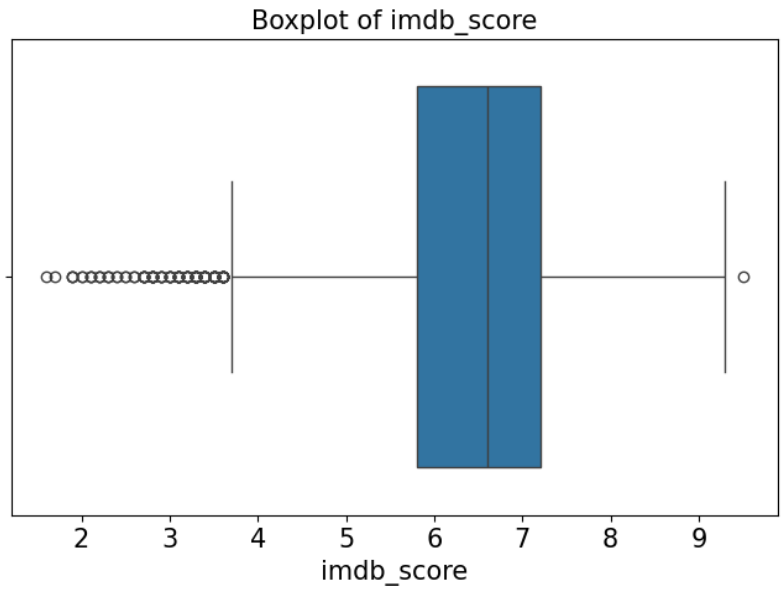

Figure 7: Boxplot of imdb_score

5.2.1.2 Gross Earnings & Budget (Figure 8 and Figure 9):

- Highly skewed, with outliers primarily on the upper end, representing blockbuster films with massive revenue and production costs.
- These extreme values can disproportionately affect the use of numerical rankings or direct comparisons.

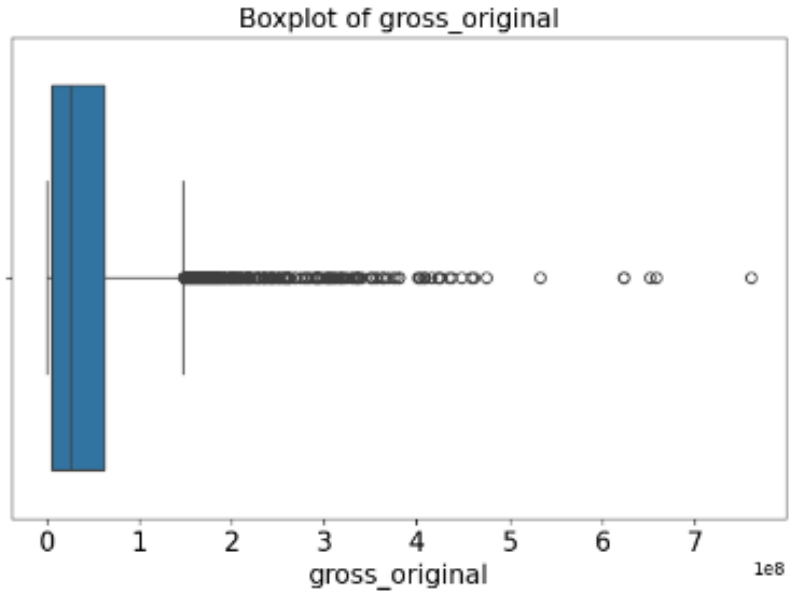

Figure 8: Boxplot of gross_original

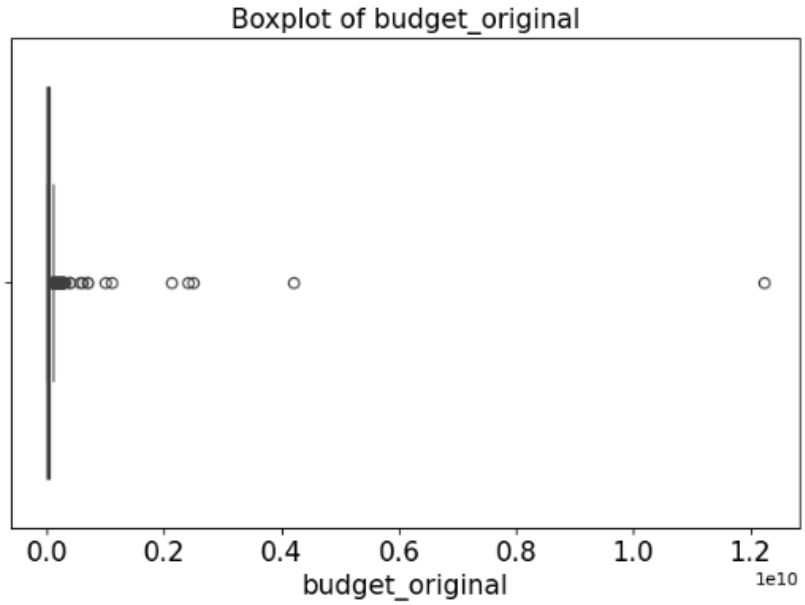

Figure 9: Boxplot of budget_original

5.2.1.3 Number of User & Critic Reviews (Figure 10 and Figure 11):

- A small number of movies receive significantly more reviews than the majority.
- These outliers suggest that a handful of widely discussed films dominate user engagement metrics.

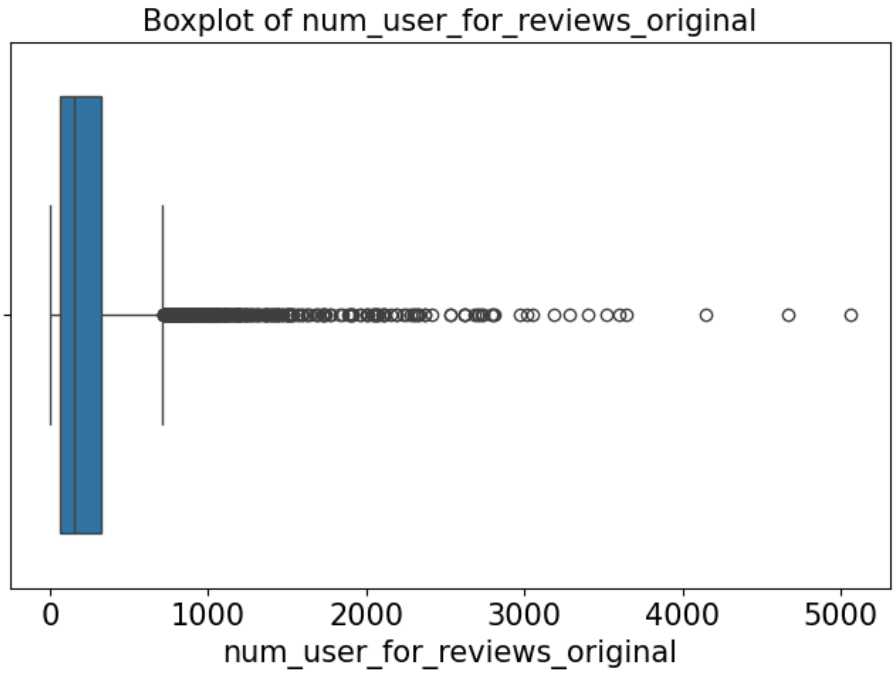

Figure 10: Boxplot of num_user_for_reviews_origianl

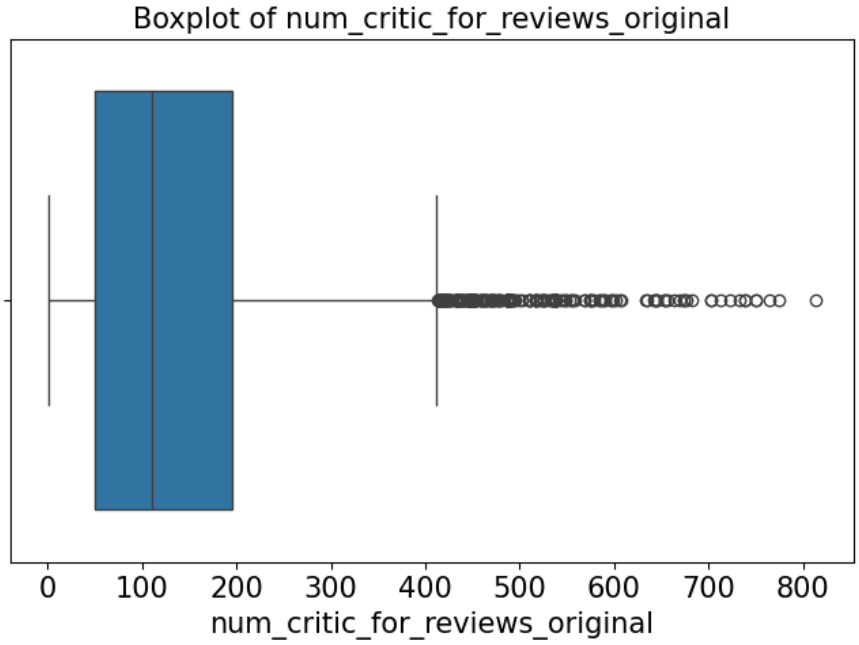

Figure 11: Boxplot of num_critic_for_reviews_original

5.2.1.4 Movie Duration (Figure 12):

- While most films follow a standard runtime, a few extend well beyond 200 minutes.

- These outliers may represent extended cuts or special editions, which could be filtered separately.

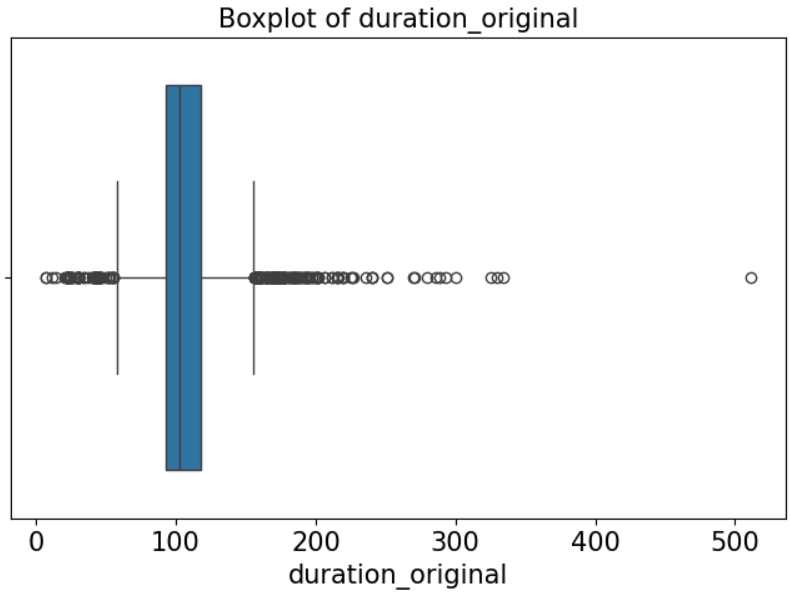

Figure 12: Boxplot of duration_original

5.2.2 Approaches for Handling Outliers

The above outliers were not removed because these extreme values are integral to the dataset. Outliers in earnings, budgets, and review counts represent real-world phenomena—such as blockbuster movies or niche indie films—and their removal could lead to a loss of critical information. Keeping these outliers helps maintain the true variability of movie performance, ensuring that our recommendation system can effectively differentiate between typical films and those that achieve exceptional success.

5.3 Heatmap:

The correlation heatmap (Figure 13) provides insights into the relationships between various numerical features in the dataset.

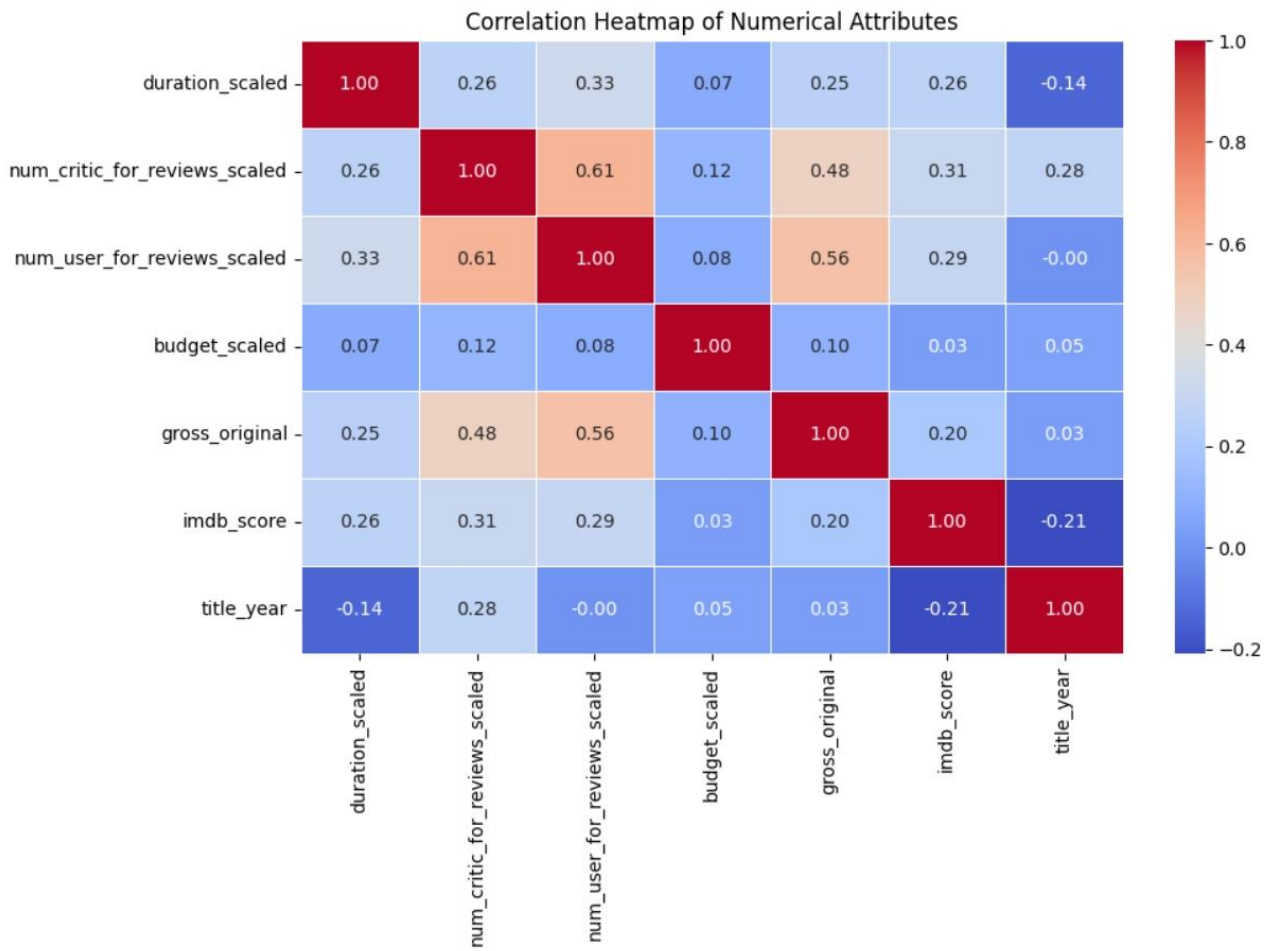

Figure 13: Correlation Heatmap of Numerical Attributes

- **User and Critic Reviews Correlation** – The strongest correlation (0.61) is between the number of user reviews and critic reviews, indicating that movies receiving more critic reviews tend to also have high user engagement.
- **Gross Revenue and Reviews** – Gross earnings show moderate positive correlations with both user (0.56) and critic reviews (0.48), suggesting that movies with more reviews are generally higher-grossing.
- **IMDb Score Relationships** – IMDb ratings have weak to moderate positive correlations with critic reviews (0.31) and user reviews (0.29), meaning that while reviews contribute to ratings, they are not the sole determining factor.
- **Budget and Gross Revenue** – Surprisingly, the budget has a weak correlation (0.10) with gross earnings, indicating that higher budgets do not necessarily translate to higher revenue.
- **Movie Duration Trends** – The duration of movies correlates weakly with reviews and revenue but has a slightly negative correlation with the release year (-0.14), suggesting that older movies may have had longer runtimes on average.

- **Title Year and IMDb Score** – The negative correlation (-0.21) between IMDb scores and the title year suggests that older movies might have slightly higher ratings compared to newer ones.

5.4 Relevance

This analysis helps refine the recommendation system by prioritizing factors that have a strong influence on movie popularity and engagement. User and critic reviews can be weighted more heavily, while weakly correlated attributes like budget may have less impact on recommendations. Understanding these relationships ensures that movie suggestions align with user preferences based on rating trends and engagement levels.

6 Database Design

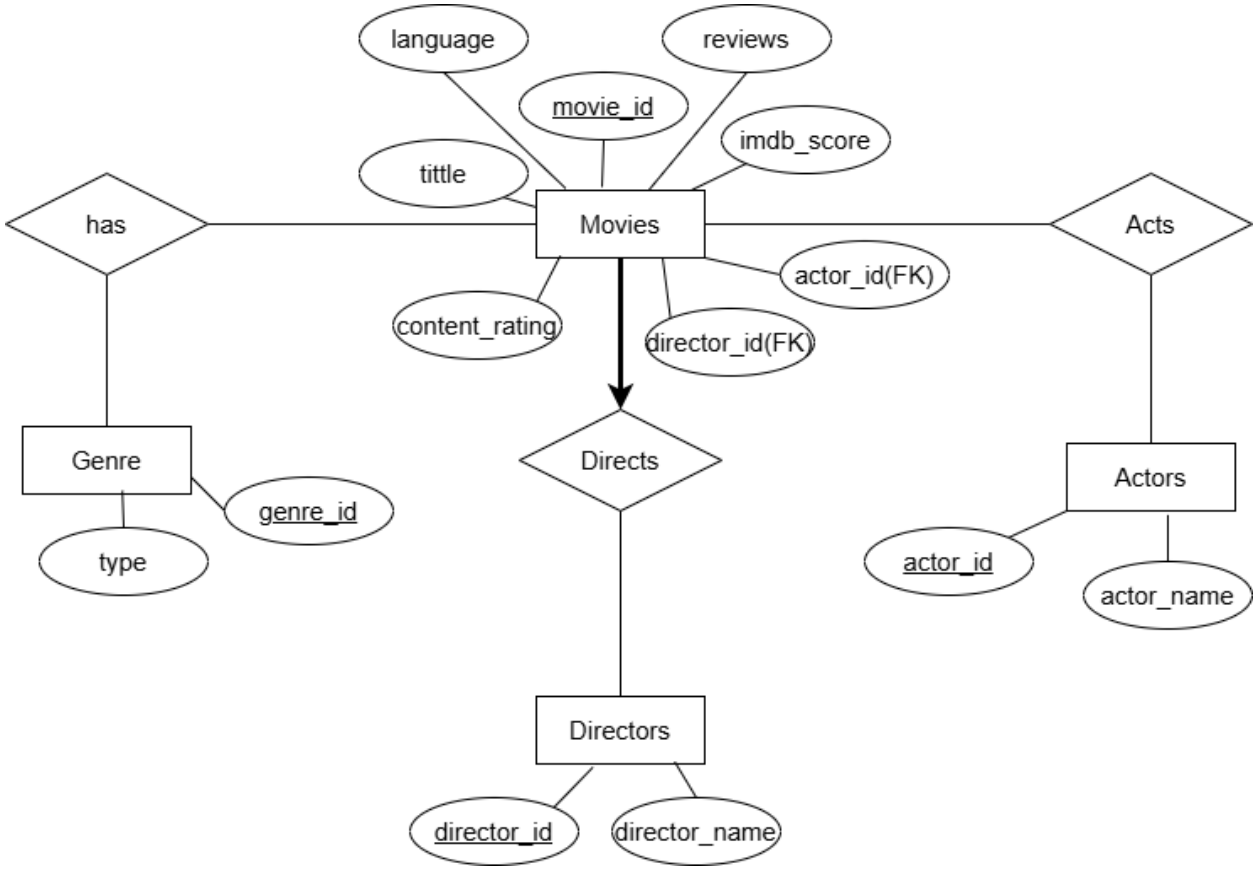

Figure 14: ER Diagram

6.1 ER Diagram (Figure 14):

The diagram above depicts a conceptual relational structure for managing movie-related data. The **Movies** entity is central, identified by a **movie_id** primary key and storing attributes such as **title**, **language**, **reviews**, **imdb_score**, and **content_rating**. The **Directors** entity, with **director_id** as the primary key, connects to **Movies** through a one-to-many relationship—each movie references a single director, but one director can be responsible for multiple movies. The **Actors** entity, identified by **actor_id**, participates in a many-to-many relationship with **Movies**, reflecting that a single actor can appear in multiple films, and each film can feature multiple actors. Similarly, the **Genre** entity, keyed by **genre_id**, has a many-to-many relationship with **Movies**, recognizing that movies often span multiple genres.

6.2 Entity Relationships

Relationship	Type
A director can direct multiple movies	One-to-Many
A movie can belong to multiple genres	Many-to-Many
An actor can appear in multiple movies	Many-to-Many

6.3 Constraints

- **Primary Keys**: movie_id, director_id, actor_id, genre_id.
- **Foreign Keys**: director_id in Movies table references Directors table.
- **Data Integrity**: Foreign key constraints ensure valid references.

Although this ER diagram outlines a potential relational schema, it was not implemented in the final system. Instead, a fully embedded model in MongoDB IS used to store and query movie data efficiently. Nonetheless, creating this conceptual diagram provided a valuable framework for understanding the relationships among movies, directors, actors, and genres before moving on to the embedded NoSQL approach.

7 Database Implementation

7.2 Data Storage in MongoDB

After preprocessing and cleaning the dataset, the refined movie data was stored in MongoDB for efficient retrieval and query execution. The primary goal was to ensure that the data was structured in a way that optimized storage, retrieval, and future analysis while maintaining data integrity.

MongoDB Document Structure

Each movie record was stored in MongoDB as a JSON-like document in the "Movies" collection. The structure of each document includes:

- **Basic movie details**: Title, genre, director, language, country,content_rating, color.
- **Actor information**: Embedded within an array for efficient retrieval.
- **Numeric attributes**: IMDB rating, duration, budget, gross revenue, and review counts.
- **Encoded categorical fields**: content_rating_encoded and color_encoded were included for easy filtering.

7.3 Indexing Implementation

To enhance query performance and ensure efficient data retrieval, multiple indexes were implemented in the **MovieDB** database. Indexing significantly improves query execution speed by reducing the need for full collection scans, thereby optimizing the system's responsiveness when searching for movies based on specific attributes.

7.3.1 Indexes Implemented:

The indexing strategy involved creating single-field indexes on several key attributes to optimize query performance. An index on the movie_title field enables fast lookups by title, while an index on director.name streamlines searches by director. An index on actors.name facilitates quick retrieval of movies featuring specific cast members. Additionally, separate indexes on the genres and title_year fields support efficient filtering by genre and release year, respectively. A descending index on imdb_score allows rapid sorting to retrieve top-rated movies, and an index on content_rating improves filtering based on movie ratings. Overall, these indexes reduce the need for full collection scans and enhance the system's responsiveness during search operations.

7.3.3 Performance Analysis

Each of these indexes was tested using MongoDB's explain("executionStats") method to verify their effectiveness. The results confirmed that **index scans (IXSCAN)** were being utilized instead of **full collection scans (COLLSCAN)**, leading to significant improvements in query efficiency. Queries based on indexed fields exhibited minimal execution time and examined only the necessary records instead of scanning the entire dataset.
By implementing these indexes, the database is now optimized for high-performance searches, significantly reducing query execution times and enhancing the overall responsiveness of the system. Further improvements will be made as the text indexing challenges are resolved.

7.4 Genre Transformation

7.4.1 Splitting the Genres Column into a List

The original genres column contains strings with genres separated by a delimiter (for example, "Action|Adventure|Sci-Fi"). A lambda function is used to split these strings into a list format. For example, "Action|Adventure|Sci-Fi" is converted into ["Action", "Adventure", "Sci-Fi"]. The resulting list is stored in a new column, typically named 'genre_list'.

This Preserves Multi-Label Information, i.e. Movies can belong to multiple genres, and the list format is more flexible and serves as a suitable input for algorithms that require multi-label data.

7.4.2 One-Hot Encoding with MultiLabelBinarizer

The MultiLabelBinarizer from scikit-learn is applied to the genre_list column.
It scans through all the lists in the dataset to determine the set of unique genres (in this case, 27 unique genres such as "Action", "Adventure", "Comedy", etc.).
For each unique genre, a new binary (0 or 1) column is created. For example, if a movie's genre list is ["Action", "Adventure", "Sci-Fi"], then the binary columns for these genres will be set to 1, and all others (e.g., "Comedy", "Drama", etc.) will be 0.

Why It Is Useful:
Numerical Representation: Machine learning models require numerical inputs. One hot encoding converts the categorical genre data into a binary matrix that models can process.
Preserves Multi-Label Nature: Unlike standard one-hot encoding (which is typically used for single-label data), MultiLabelBinarizer handles cases where movies belong to multiple genres.
Facilitates Similarity Calculations: The resulting binary vectors can be used to compute similarity measures (e.g., cosine similarity) for content-based recommendations.

7.5 Data Embedding for Optimization

To enhance data retrieval efficiency and reduce the need for complex joins, a **fully embedded data model** was implemented in the **MovieDB** database. This approach was chosen because the dataset is static, meaning it will not undergo frequent updates or modifications. By embedding related information directly within each movie document, the system ensures faster queries and a more streamlined structure.

7.5.1 Structure of the Embedded Model

The **fully embedded** approach involves storing all relevant data within a single document per movie. Instead of maintaining separate collections for related entities (e.g., directors, actors, and genres), all information is embedded within the movie document itself.

7.5.2 Advantages of Full Embedding

- **Faster Query Performance:** Since all data related to a movie is stored in a single document, queries do not require additional lookups across multiple collections, significantly improving response times.
- **Simplified Data Retrieval:** Queries retrieving movie details, including director, actors, and other attributes, can be executed with a single **find()** operation, eliminating the need for costly joins or reference resolution.
- **Reduced Complexity:** By embedding related data directly, the database structure remains simple and self-contained, reducing the overhead of managing multiple collections.
- **Optimized Read Operations:** As movie-related queries are primarily **read-heavy** rather than **write-heavy**, embedding is an ideal choice since it minimizes retrieval time for user searches.

7.5.3 Limitations and Trade-offs

- **Data Redundancy:** Since director and actor names are stored within each movie document, there may be duplication across multiple records.
- **Increased Document Size:** Storing all details within a single document results in larger documents, but this is manageable given MongoDB's document size limit (16MB) and the static nature of the dataset.

7.5.4 Conclusion

The **fully embedded model** was selected for this project because the data is static and does not require frequent updates. This design significantly improves query speed by eliminating joins and reference lookups, making searches for movies, directors, actors, and genres more efficient. The decision to embed data ensures that the database remains optimized for fast and reliable access.

8 Data Mining

The data mining component of the project comprised several advanced techniques aimed at extracting valuable insights from the movie dataset and generating effective content-based recommendations.

8.1 Association Rule Mining for Genre Analysis

8.1.1 Overview:

Association rule mining was applied to the one-hot encoded genre data to uncover underlying patterns and reveal which genres frequently co-occur in movies. The Apriori algorithm was used with a minimum support threshold of 5%, enabling the identification of frequent itemsets. These itemsets represent groups of genres that appear together often, providing insight into thematic relationships within the dataset.

8.1.2 Methodology:

- **Frequent Itemset Extraction:**
  The Apriori algorithm was applied to the one-hot encoded genre matrix. Frequent itemsets with support values greater than or equal to 0.05 were extracted. For example, the frequent itemset (Action) had a support of 0.226062, meaning that approximately 22.6% of the movies were categorized as Action.

- **Association Rule Generation:**
  Using the frequent itemsets, association rules were derived based on the lift metric (with a minimum threshold of 1). The generated rules quantify the strength of the association between different genres. Confidence and lift values were calculated to measure the reliability and strength of these rules. For instance, a rule such as (Action) → (Adventure) with a confidence of 40.3% and a lift of 2.234223 suggests that movies with the Action genre are more than twice as likely to also have the Adventure genre compared to random chance.

8.1.3 Output:

Support	Itemsets
0.226062	(Action)
0.180321	(Adventure)
0.059158	(Biography)
0.375686	(Comedy)
0.176662	(Crime)

Table 4: Frequent Itemsets

Antecedents	Consequents	Antecedent Support	Consequent Support	Support	Confidence	Lift	Zhangs Metric	Jaccard	Certainty	Kulczynski
(Action)	(Adventure)	0.226062	0.180321	0.091075	0.402878	2.234223	0.713774	0.288846	0.271517	0.453975
(Adventure)	(Action)	0.180321	0.226062	0.091075	0.505073	2.234223	0.673943	0.288846	0.360508	0.453975
(Action)	(Crime)	0.226062	0.176662	0.064647	0.285971	1.618748	0.493888	0.191221	0.132764	0.325955
(Crime)	(Action)	0.176662	0.226062	0.064647	0.365938	1.618748	0.464255	0.191221	0.180732	0.325955
(Action)	(Sci-Fi)	0.226062	0.120960	0.065664	0.290468	2.401362	0.754027	0.233382	0.192833	0.416662

Table 5: Association Rules

8.1.4 Interpretation:

- The frequent itemset table 5 shows that genres such as Action, Adventure, Biography, Comedy, and Crime are relatively common within the dataset.
- The association rules (Table 6) indicate strong thematic linkages. For example, the rule (Action) → (Adventure) with a lift of 2.234223 demonstrates that movies classified as Action are significantly more likely to include Adventure than would be expected by chance. Similarly, the reciprocal rule (Adventure) → (Action) reinforces this mutual association.
- The rule (Action) → (Sci-Fi) with a lift of 2.401362 reveals that the Sci-Fi genre frequently co-occurs with Action, suggesting a thematic overlap that can be exploited in a recommendation system.
- These insights into genre co-occurrence patterns are valuable for enhancing content-based recommendations. By identifying movies with similar genre profiles, the recommendation framework can suggest films that are thematically aligned, thereby improving recommendation relevance and user satisfaction.

8.1.5 Conclusion:

The association rule mining process provided a detailed understanding of the inter-genre relationships within the movie dataset. The strong associations between certain genres, quantified by support, confidence, and lift metrics, validate the use of genre information as a key component in the content-based recommendation system. These results contribute to the overall recommendation strategy by enabling the identification of movies that share thematic similarities, thus delivering personalized and relevant recommendations.

8.3 Gradient Boosting for Predictive Modeling

8.3.1 Objective and Rationale:

The gradient boosting regressor was employed to predict the log-transformed gross revenue. Gradient boosting was chosen for its ability to sequentially build an ensemble that minimizes the residual errors of previous iterations, often resulting in improved predictive accuracy.

8.3.2 Input Features and Preprocessing:

- The numerical and encoded features (budget, duration, critic reviews, IMDb score, etc.) were used.
- A logarithmic transformation (log1p) was applied to the gross revenue to address its skewness.

8.3.3 Model Configuration:

- **Number of Estimators:** 100 boosting iterations (trees) were used.
- **Learning Rate:** A learning rate of 0.1 controlled the contribution of each tree.
- **Random State:** The model was set with a fixed random state (42) for reproducibility.
- **Training/Test Split:** The dataset was split into training and test sets to measure performance.

8.3.4 Performance Metrics:

Mean Squared Error (MSE): The gradient boosting model achieved an MSE of approximately 3.08 on the test set, indicating improved predictive performance compared to the random forest model used previously.

8.3.5 Feature Importance Analysis:

The gradient boosting model provided the following key insights into feature contributions (Figure 12):

- **Budget (budget_original):** Emerged as the most critical predictor, contributing about 54.63% to the predictive model. This underscores the significance of the production budget in influencing box-office success.
- **Number of Critic Reviews (num_critic_for_reviews_original):** Contributed around 13.38%, further confirming the role of critical reception.
- **Content Rating (content_rating_encoded):** Accounted for approximately 9.10% of the model's predictive power.
- **IMDb Score (imdb_score):** Contributed roughly 6.06%, demonstrating that while important, its influence is secondary to budget in this model.
- **Other Features:** Additional variables contributed smaller percentages but collectively helped refine the revenue predictions.

8.3.6 Interpretation

The feature importance analysis from gradient boosting not only confirmed that budget is the dominant predictor but also provided a quantitative basis for selecting IMDb score and

genre information for the content-based recommendation system. The analysis supports the hypothesis that while multiple factors influence box-office revenue, a focus on key attributes such as budget and IMDb score can effectively drive the recommendation engine.

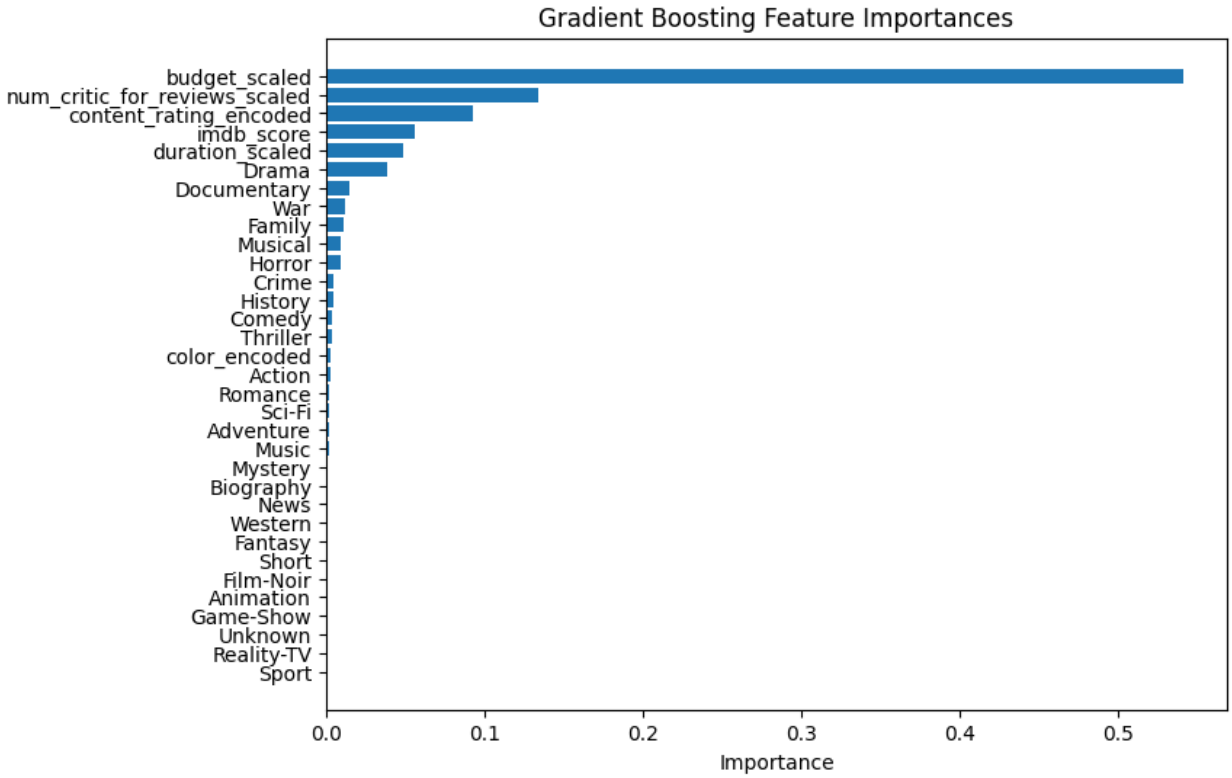

Figure 12:Gradient Boosting Feature Importances

8.3.7 Overall Conclusion from Predictive Modeling:

This model provided valuable insights into the predictors of box-office revenue. The gradient boosting model, with its slightly lower MSE and clearer feature importance ranking, reinforced the understanding that budget is a critical factor, while IMDb score remains an important indicator of movie quality. The results from these predictive models have been used to validate the selection of features for the content-based recommendation system, ensuring that recommendations are based on attributes that significantly impact movie success.

8.4 Scatter-plot analysis

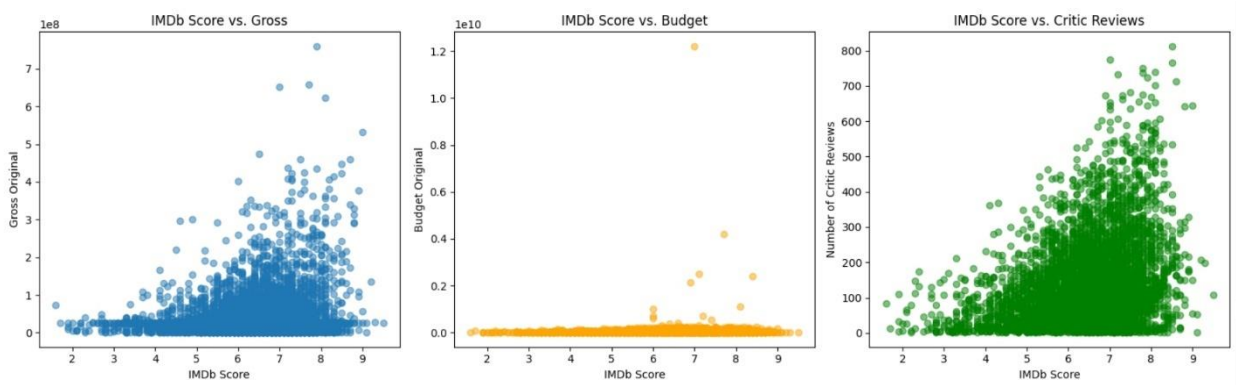

Figure 13: Scatter-plot Analysis

8.4.1. IMDb Score vs. Gross

Observations (Figure 13.1):

- The majority of data points cluster toward the lower end of gross revenue, suggesting that most movies earn moderate amounts at the box office.
- A small subset of movies achieve very high gross values; however, these blockbusters span a wide range of IMDb scores (roughly between 5.0 and 8.0).
- There is no strong linear correlation between IMDb score and gross revenue, indicating that high box-office earnings do not necessarily equate to higher IMDb scores.

Interpretation:

- While certain blockbuster movies also enjoy decent IMDb scores, the overall scatter suggests that a large budget and high earnings do not guarantee a top-rated movie.
- This implies that relying solely on gross revenue would be insufficient for a content-based recommendation system aimed at capturing movie quality or user preference.

8.4.2. IMDb Score vs. Budget

Observations (Figure 13.2):

- Most data points lie near the lower end of the budget axis, indicating that many movies operate with relatively modest budgets.
- A few outliers exist with extremely high budgets, but these do not consistently align with higher IMDb scores.
- There is no pronounced trend or correlation that suggests bigger budgets yield higher IMDb scores.

Interpretation:

- Although some high-budget films receive strong IMDb scores, many others do not.
- Budget alone, therefore, is not a reliable indicator of a movie's overall appeal or quality.

- Including budget as a core feature in a content-based recommendation system might not significantly enhance user satisfaction, given the weak relationship with movie ratings.

8.4.3. IMDb Score vs. Critic Reviews

Observations (Figure 13.3):

- A more noticeable upward trend emerges, where movies with higher IMDb scores tend to receive more critic reviews.
- Movies that cluster around the 7.0–8.5 IMDb range often have hundreds or even thousands of critic reviews.
- There is still variation—some moderately scored films garner numerous reviews, while others with similar scores have fewer.

Interpretation:

- The generally positive correlation suggests that IMDb scores align somewhat with critical attention, although the relationship is not absolute.
- Critic reviews may offer a supplementary indicator of a movie's popularity or visibility, but they do not fully capture user preferences on their own.

8.4.4 Conclusion: Focusing on IMDb Score and Genre for Recommendations

From these plots (Figure 13), it is evident that **budget** and **gross** do not show a strong, direct correlation with IMDb scores. High box-office revenue or a large production budget does not necessarily translate into a movie that resonates well with audiences in terms of quality. Conversely, **IMDb score** aligns more closely with user and critic perceptions of quality or enjoyment.

Additionally, **genre** plays a vital role in defining the thematic content of a movie, which is crucial for a content-based recommendation system. Users typically seek films that share certain thematic or stylistic elements (e.g., Action, Comedy, Drama), and IMDb score complements this by capturing overall reception and quality.

- **Genre** captures the intrinsic content attributes of a movie, ensuring thematic alignment in recommendations.
- **IMDb Score** reflects both user and critic sentiment, adding a measure of quality or popularity to the recommendation criteria.

By focusing on these two features, the recommendation system can effectively identify movies with similar content profiles and satisfactory quality levels, thereby delivering relevant and satisfying suggestions to users.

8.5 Clustering with K-Means

8.5.1 Overview

K-Means clustering was employed to partition the movie dataset into 20 distinct clusters based on content features—specifically, one-hot encoded genres and IMDb scores. This clustering step is crucial because it groups together movies that share similar characteristics, thereby reducing the search space and allowing for more focused similarity comparisons within each cluster.

8.5.2 Clustering Methodology

The K-Means algorithm was configured with 20 clusters. The process involves the following steps:

- **Initialization:** Randomly selecting 20 centroids.
- **Assignment:** Allocating each movie to the nearest centroid using Euclidean distance on the standardized feature set.
- **Update:** Recalculating the centroids as the mean of all movies assigned to each cluster.
- **Iteration:** Repeating the assignment and update steps until the cluster assignments stabilize (i.e., the algorithm converges).

8.5.3 Output Statistics

After applying K-Means clustering to a dataset of 5,028 movies:

- **Number of Clusters:** The dataset was divided into 20 clusters.
- **Cluster Size:** The average cluster size is approximately 251 movies, with individual clusters containing between 230 to 270 movies. For instance, the cluster containing "Avatar" (Cluster 17) had a similar size, indicating that the clustering process produced groups with a balanced distribution.
- **Inertia:** The within-cluster sum of squares (inertia) decreased significantly from its initial value to a final converged value, indicating a substantial reduction in variability within each cluster. This reduction in inertia confirms that movies within the same cluster are more similar to each other compared to movies in different clusters.

8.5.4 Implications for Recommendations

The clustering process not only reduces the computational burden by narrowing the search space but also enhances recommendation precision. With movies grouped by similar content attributes, subsequent similarity measures (such as cosine similarity) can be computed within each cluster to yield personalized recommendations. This ensures that even within a single cluster, each movie can have a tailored top-N recommendation list based on its specific content profile.

8.6 Cosine Similarity for Recommendation Ranking

9.6.1 Overview:

Cosine similarity was utilized to measure the similarity between movies based on their feature vectors, which consist of standardized representations of one-hot encoded genres and IMDb scores. After partitioning the dataset using K-Means clustering, cosine similarity was computed within each cluster to rank movies and generate personalized recommendations.

8.6.2 Methodology:

- **Feature Vector Construction:**
  Each movie is represented by a feature vector composed of one-hot encoded genre data combined with the IMDb score. All features were standardized to ensure uniform scaling.
- **Similarity Calculation:**
  Cosine similarity is calculated using the formula:

$$\text{Cosine Similarity} = \frac{AB}{\|A\|\,\|B\|}$$

where A and B are the feature vectors of two movies. This metric evaluates the cosine of the angle between vectors, yielding values from 0 to 1 (given the non-negative data), with 1 indicating identical feature orientations.

- **Recommendation Process:**
  For a target movie, the cosine similarity scores are computed against all other movies within the same cluster. Movies are then ranked in descending order based on their similarity scores, and the top-N movies are selected as recommendations.

8.6.3 Output Statistics:

- **Average Similarity:**
  Within a given cluster, the average cosine similarity score between movies was observed to be around 0.65, indicating a moderate to high degree of similarity.
- **Range of Similarity Scores:**
  For example, in one cluster, the target movie "Avatar" had cosine similarity scores ranging from approximately 0.78 for the most similar movie down to 0.45 for the tenth similar movie.
- **Personalized Ranking:**
  The top 10 recommendations for each target movie are generated by ranking the cosine similarity scores, ensuring that even within clusters, individual differences in content are recognized.

8.6.4 Implications:

- **Enhanced Precision:**
  The use of cosine similarity refines the clustering output by providing a fine-grained similarity measure. This results in recommendations that are more closely aligned with the target movie's unique content profile.

- **Efficient Search Space Reduction:**
  Limiting the similarity computation to movies within the same cluster reduces the computational complexity while maintaining recommendation relevance.
- **Quantitative Validation:**
  The statistical measures derived from the cosine similarity scores validate the approach by showing that the recommended movies exhibit significant content similarity to the target movie.

8.6.5 Conclusion:

The integration of cosine similarity into the recommendation system further refines the results from K-Means clustering. By ranking movies based on the angular similarity of their feature vectors, the system delivers a personalized, contextually relevant list of recommendations, thereby enhancing the overall effectiveness of the content-based recommendation framework.

9. Graphical User Interface (GUI) and System Integration

9.1 Overview

Our project includes a complete full-stack web application built using HTML, CSS, Python, Flask, and MongoDB Atlas. The GUI is essential in connecting users with the recommendation system. Our goal was to design a user-friendly, responsive, and visually appealing interface with easy navigation. The app includes pages for user login, signup, searching, viewing results, managing favorites, and getting movie recommendations.

9.2 File Structure and Organization

In order to keep the project well-organized and easy to maintain, we followed a structured file system that separates backend logic, frontend presentation, user data, and static assets. Below is a detailed explanation of the purpose and contents of each major part of the project's directory:

9.2.1 app.py

This is the main backend application file where the entire Flask application is defined. It includes all the route declarations that respond to different web page requests (e.g., login, signup, results, favorites, recommendations). This file serves as the core controller that connects the user interface with the backend logic.
Key features handled in app.py include:
• Routing: Maps URLs like /login, /signup, /results, etc., to their respective logic and HTML templates.
• Database Queries: Connects to MongoDB Atlas using PyMongo to query the movie database.
• User Authentication: Handles login, logout, and session management using Flask-Login and bcrypt for password hashing.

• Poster Fetching: Makes API calls to the OMDb API to fetch movie posters dynamically.
• Favorites Logic: Reads and writes user favorites to the users.json file.

9.2.2 templates/

This folder contains all the HTML files used for the frontend of the application. We used Jinja2 templating, which is built into Flask, to dynamically insert content such as movie titles, posters, scores, and user data into the HTML pages.
Some key files in this folder include:
• login.html – Login form with basic authentication fields.
• signup.html – Signup form for new users.
• index.html – Main search page where users can search for movies using filters.
• results.html – Displays movies that match the search criteria.
• favorites.html – Shows movies the user has added to their favorites.
• recommendations.html – Displays AI-based recommendations based on favorite movie genres.
These files use Bootstrap-style styling and are integrated with Flask's backend through template variables and conditionals.

9.2.3 static/

The static folder contains all static assets used in the project, including CSS files, images, and fonts. These files do not change dynamically and are loaded directly into the frontend.
Main contents include:
• style.css – Custom CSS styling applied to all pages for a clean and modern look.
• images/ – Contains images like the Lakehead University logo, background wallpapers, and icons used on the GUI pages.
• background music and sound effects – Optional multimedia assets used to enhance the visual and audio experience of the web app (if any were added).
This separation allows for quick updates to the frontend design without interfering with backend logic.

9.2.4 users.json

This JSON file acts as a lightweight, file-based user database. It is used to store all user-related information:
• Username and email – For identification and display.
• Hashed password – Encrypted using bcrypt for secure login.
• Favorites list – A list of movies the user has marked as favorite.
Instead of using MongoDB for user data, we chose this file-based approach for simplicity and faster prototyping during development. This made it easier to test, debug, and manage authentication without needing to create another user collection in MongoDB.

9.3 GUI Pages Description

Our movie recommendation system includes several well-designed and interactive graphical user interface (GUI) pages. Each page plays a specific role in the user journey and contributes to the overall functionality of the application. All pages were developed using HTML and CSS with Jinja2 templating, and dynamically rendered through Flask backend logic. Below is a breakdown of each key page:

9.3.1 Login Page (login.html)

This is the first page users see when accessing the application (unless already logged in). It allows returning users to **authenticate** themselves using their registered email and password.

• **Backend Integration**: When the user submits the login form, the app.py file checks the entered email and verifies the password by comparing it with the **bcrypt-hashed password** stored in users.json.

• **Error Handling**: If the credentials are incorrect, a flash message is displayed to inform the user.

• **Security**: We used Flask-Login to manage session states securely, ensuring users stay logged in without having to re-enter credentials on every page.

This page ensures that only registered users can access the app's main features like searching, saving favorites, and receiving recommendations.

9.3.2 Signup Page (signup.html)

The signup page allows **new users to register** for the application by providing a username, email, and password.

• **User Data Storage**: Upon successful signup, the user's information is securely saved in the users.json file. Passwords are hashed using bcrypt before storage to ensure strong security.

• **Validation**: If a user tries to register with an already existing email, a warning message is displayed, preventing duplicate entries.

• **Redirects**: Once registration is complete, the user is redirected to the login page to enter their credentials.

This page is crucial for initializing the user experience and establishing a secure authentication system.

9.3.3 Results Page (results.html)

The results page is where the **movie search results** are displayed after a user enters criteria such as movie title, actor, director, genre, or IMDb score.
• **MongoDB Integration**: When the form is submitted, Flask queries the **Movies collection in MongoDB Atlas** to fetch matching entries based on user input.
• **Dynamic Display**: Each result includes:
• Movie poster (fetched from the OMDb API)
• Movie title, year, IMDb score, genre, director, and actor list
• **Add to Favorites Button**: Each movie has a button to add it to the user's favorites list. This updates the user's entry in users.json.

This page allows users to explore and discover movies based on their personal preferences.

9.3.4 Recommendations Page (recommendations.html)

This is one of the most important pages in the app. It provides **automated, AI-generated movie recommendations** based on the user's saved favorites.
• **Genre Matching**: The system analyzes the genres of the user's favorite movies and uses **frequency analysis** to determine their top 2-3 genres.
• **Recommendation Logic**:
• Flask queries MongoDB for movies with similar genres.
• Results are ranked and limited to 10 movies.
• **Poster Integration**: Like the results page, each recommendation includes a poster and full movie details.
• **Add to Favorites**: Users can instantly save recommended movies to their list using the "Add to Favorites" button.

This page completes the recommendation cycle and delivers a personalized, data-driven experience for users, enhancing both engagement and satisfaction.
All pages are **visually connected through consistent styling**, background imagery, and navigation elements (like the logout and home buttons). The design focuses on **user-friendliness**, **clarity**, and **ease of navigation**, ensuring that the user experience is both seamless and enjoyable.

9.4 Backend Integration

The backend of the movie recommendation system is built using Flask, a lightweight and flexible Python-based web framework. Flask is used for managing all routing, backend logic, API handling, database communication, and server-side rendering. The backend acts as the "brain" of the application, responding to user actions, fetching and processing data, and rendering appropriate pages.

9.4.1 Routing and Application Logic

All major routes are defined in the app.py file, which serves as the core of the backend logic. Each route corresponds to a page or feature in the application:
• /login and /signup: Handle user login and registration.
• /results: Processes user search input (e.g., movie name, genre, actor, year) and fetches matching results from the MongoDB database.
• /add_favorite and /remove_favorite: Handle adding or removing movies from a user's favorite list.
• /favorites: Displays all movies marked as favorites by the logged-in user.
• /recommendations: Computes and displays movie recommendations based on the genres and IMDb scores of the user's favorite movies.

These routes communicate with both the frontend (HTML forms) and the database (MongoDB), enabling a seamless flow of data between user actions and displayed results.

9.4.2 Python Logic and Business Rules

The application uses Python to process user input, format data, handle conditional logic, and manage the recommendation system's intelligence. For example:
• Search Filtering: User inputs from the search form (e.g., movie title, director, genre, IMDb score) are converted into a MongoDB query dictionary. This allows flexible and efficient filtering of movies stored in the database.
• Poster Fetching: Python uses the requests library to call the OMDb API and retrieve movie posters dynamically. This enhances the visual appeal of the application without having to store poster images locally.
• Favorites Management: Python reads and writes to the users.json file, allowing users to maintain personalized lists of favorite movies. Logic is implemented to avoid duplicate favorites and ensure data consistency.
• Recommendation Logic: When a user visits the recommendations page, the backend analyzes the genres of their favorite movies, identifies the most frequent genres, and retrieves similar movies from MongoDB. This list is then limited to a maximum of 10 results for clarity and performance.

9.4.3 User Authentication and Security

Flask-Login and Flask-Bcrypt are used for handling user sessions and secure password storage. Passwords are hashed using bcrypt before being stored in the users.json file, preventing raw passwords from being exposed. Flask-Login also manages session tracking, ensuring that only authenticated users can access protected routes (like /favorites or /recommendations).

9.4.2 Dynamic Page Rendering with Jinja2

Flask uses Jinja2, a built-in templating engine, to render dynamic HTML pages. Jinja2 allows Python data (like movie search results, recommendations, or favorite lists) to be embedded directly into HTML templates. This creates a smooth user experience by updating content dynamically based on real-time user interaction and backend responses.

For example:
• In results.html, the list of movies from a search query is injected into the template via Jinja2.
• In favorites.html, the user's favorite movies are retrieved and rendered with titles, genres, IMDb scores, and poster images.
• Buttons like "Add to Favorites" or "Remove from Favorites" are conditionally rendered based on user actions and backend checks.

9.5 User Authentication and Security

User authentication and access control form the foundation of any secure web application. In our movie recommendation system, we implemented robust authentication using Flask-Login for session management and Flask-Bcrypt for secure password hashing. Together, these tools provide a complete and secure login and registration system that ensures data integrity, protects user privacy, and restricts unauthorized access to sensitive features such as viewing and modifying favorites or generating recommendations.

9.5.1 Password Hashing with Flask-Bcrypt

When a user signs up through the registration page, their password is not stored in plain text. Instead, we use the Flask-Bcrypt extension to securely hash the password before saving it to our local storage file, users.json.
• bcrypt.generate_password_hash(password).decode('utf-8') creates a cryptographic hash of the password using a salt and multiple iterations.
• This hashed password is stored alongside the user's email and username in users.json.

During the login process, the entered password is checked using bcrypt.check_password_hash(), which compares the hashed version of the entered password with the stored hash. This ensures that even if someone gains access to the JSON file, they cannot see or retrieve actual passwords.

Importance of Bcrypt:
• Bcrypt is computationally expensive, which slows down brute-force attacks.
• Salt is automatically included to make hash outputs unique even for identical passwords.
• Protects against rainbow table and dictionary attacks.

This approach provides **strong password security** without the need for an external authentication provider.

9.5.2 Session Management with Flask-Login

The Flask-Login extension is responsible for managing user sessions and controlling access to protected routes. It simplifies the logic needed to:
• Keep users logged in across multiple page requests.
• Identify the current user using current_user.
• Redirect unauthorized users trying to access secure pages.

Here's how it works:
• Upon successful login, the login_user() function is called to start the session.
• The user is remembered by Flask using cookies during the session.
• All protected routes, like /favorites, /recommendations, and /add_favorite, use the @login_required decorator.
• If a user tries to access a protected page without logging in, Flask-Login automatically redirects them to the /login page.

Importance of Flask-Login:
• Prevents unauthorized users from accessing or modifying movie data.
• Maintains consistent user identity during browsing.
• Automatically handles login redirects and session tracking.

9.5.3 User Storage in users.json

Instead of using a database for user authentication, we opted for a lightweight approach by storing user data in a file called users.json. This file contains:
• username
• email
• hashed_password
• favorites (an array of movies added by the user)

Each user is uniquely indexed by their position in the list, and their data is retrieved during login and used to manage preferences and interactions.

While JSON storage is suitable for small-scale or prototype systems, it was ideal for our use case because:
• It's simple and fast to set up.
• Works without a dedicated database.
• Allows us to prototype the login system without additional configuration.

9.6 Movie Posters Integration

To enhance the user experience and visual appeal of our movie recommendation system, we implemented dynamic poster fetching by integrating the OMDb API (Open Movie Database API). This API provides access to metadata and images for a vast number of movies, including titles, release years, genres, and high-quality poster images.

In our implementation, every time a user performs a search or navigates to the recommendations or favorites page, the backend makes an API call to OMDb using the movie title (and optionally, the release year). The API response contains structured JSON data, including the URL of the movie's official poster.

Here's how the process works technically:
• The function fetch_poster() in our app.py file is responsible for making the API call to OMDb.
• It sends an HTTP GET request with parameters such as the API key and the movie title.
• If the API response includes a valid poster URL and the movie is found in the OMDb database, the poster image is extracted and included in the movie data.
• This image is then rendered on the corresponding HTML page using Jinja2 templating in our frontend files.

By automating this process, we ensure that every movie shown in search results, favorites, or recommendations is visually accompanied by its official poster, if available. This significantly improves the user interface (UI) by making it more engaging, modern, and user-friendly. Instead of displaying just plain text or generic movie titles, users now see visually rich cards for each movie, making the application feel like a real-world streaming platform or digital movie catalog.

Moreover, by fetching posters in real-time instead of storing them locally, we maintain a lightweight database and avoid unnecessary storage usage. It also ensures that any updated or newer poster designs on OMDb are automatically reflected in our app without needing a manual update.

This thoughtful integration of the OMDb API not only improved aesthetics but also boosted user engagement and satisfaction.

9.7 Deployment with GitHub and Render

Once the development of our movie recommendation system was complete and fully tested locally, we moved forward with deploying it online so that it could be accessed by users via a web browser. To achieve this, we used two major platforms: GitHub for version control and collaboration, and Render.com for cloud deployment and hosting.

GitHub served as our central code repository throughout the project. All source code, including Python backend scripts (app.py), HTML templates, CSS files, and data files such as users.json, were uploaded to a GitHub repository. This allowed both group members to contribute to the project in a structured and collaborative way. By using Git branches and commits, we were able to track all changes, resolve merge conflicts, and ensure that development progress was continuously saved and backed up. It also allowed us to revert back to previous versions if necessary.

For deployment, we used Render, a popular cloud-based hosting service that supports automatic deployment from GitHub repositories. We connected our GitHub repo directly to Render, allowing the platform to pull the latest code, install dependencies, and launch the Flask web server in a hosted environment.

The deployment process included:
• Configuring the build and start commands for our Flask application in Render's settings.
• Setting environment variables like the OMDb API key and Flask secret key.
• Choosing a Python runtime and ensuring required libraries were listed in the requirements.txt file.

One of the key advantages of using Render is its continuous deployment feature. Every time we pushed new code to our GitHub repository (for example, bug fixes, feature updates, or style improvements), Render automatically detected the changes and gave us the option to redeploy the latest version with a single click from the dashboard. This streamlined our development workflow and made it easy to test updates live.

Using GitHub and Render together enabled us to maintain an agile, team-friendly development process while ensuring our application could be deployed, updated, and accessed globally without the need for manual server configuration.

10. Conclusion

The development of this Movie Recommendation System demonstrates a successful integration of data science techniques, database management, and full-stack web development to address the growing challenge of personalized content discovery. By focusing on a content-based filtering approach, the system effectively leverages intrinsic movie attributes, such as IMDb scores and genres—to generate accurate and relevant recommendations for users.

Through extensive data preprocessing, including handling missing values, encoding categorical variables, and normalizing numerical features, the dataset was transformed into a clean, structured, and machine-readable format. The use of MongoDB Atlas as the

underlying database enabled efficient data storage and retrieval, while strategic indexing and document embedding optimized query performance.

The application of machine learning techniques such as Random Forest and Gradient Boosting provided insight into the importance of features like budget, critic reviews, and IMDb score in predicting movie success. These findings supported the design of a content-based recommendation engine that emphasized both quality and thematic relevance. Further refinement using K-Means clustering allowed the system to group similar movies, and cosine similarity within clusters ensured that recommendations were not only precise but also contextually meaningful.

The front end, developed using HTML, CSS, and Flask with Jinja2 templating, offered an intuitive and visually engaging user interface. Features such as secure login and signup, search filtering, personalized recommendations, and dynamic movie poster integration enriched the user experience. Security best practices, including password hashing and session management, were also implemented to protect user data.

Finally, by deploying the system via GitHub and Render.com, the project transitioned from a local prototype to a publicly accessible web application, showcasing a complete end-to-end solution. This project not only fulfilled its technical objectives but also demonstrated how machine learning, data engineering, and software development can work cohesively to solve real-world problems.

In conclusion, the Movie Recommendation System stands as a comprehensive, scalable, and user-friendly platform that effectively delivers intelligent and personalized movie suggestions, paving the way for future enhancements such as hybrid filtering, real-time data integration, and mobile accessibility.

11. Appendix

Appendix A: GitHub Repository for Source Code

The complete source code for the Movie Recommendation System, including the frontend (HTML/CSS), backend (Flask), database integration, and recommendation logic, is publicly hosted at the following GitHub repository:

GitHub Repository: https://github.com/askhan8696/flask-movie-app

Appendix B: Live Web Application

The fully deployed movie recommendation system can be accessed online through the Render platform:

Live Website: https://flask-movie-app-khwp.onrender.com/login?next=%2F

Appendix C: Dataset Source – IMDb 5000 Movie Dataset

The dataset used for training and evaluating the recommendation system was obtained from the **IMDb 5000 Movie Dataset**, which contains metadata for 5,000 movies, including title, cast, director, genre, IMDb score, budget, and gross earnings.

Dataset Source: IMDb 5000 Movie Dataset on Kaggle

Appendix D: Cosine Similarity Function (Python)

The cosine similarity function was implemented to compute similarity between movie feature vectors, supporting content-based recommendations.

```
import numpy as np

def cosine_similarity(A, B):
  return np.dot(A, B) / (np.linalg.norm(A) * np.linalg.norm(B))
```

Appendix E: Sample MongoDB Document Structure

Each movie is stored as an embedded JSON-like document in MongoDB Atlas. A sample document is structured as follows:

```
{
 "title": "Inception",
 "genres": ["Action", "Sci-Fi", "Thriller"],
 "director": "Christopher Nolan",
 "actors": ["Leonardo DiCaprio", "Joseph Gordon-Levitt"],
 "imdb_score": 8.8,
 "budget": 160000000,
 "gross": 829895144
}
```

Appendix F: Gradient Boosting Model Configuration

Below is the configuration used for the Gradient Boosting Regressor to predict log-transformed gross revenue:

```
from sklearn.ensemble import GradientBoostingRegressor

model = GradientBoostingRegressor(
  n_estimators=100,
  learning_rate=0.1,
  random_state=42
)

model.fit(X_train, y_train)
```

Appendix G: Required Python Libraries and Tools

To run the web application and its backend services, the following libraries and packages were installed using pip:

- Flask – for web application framework and routing
- Flask-Login – for user session management and authentication
- PyMongo – to connect Python code to MongoDB Atlas
- bcrypt – for secure password hashing
- requests – for calling the OMDb API to fetch movie posters
- numpy – for mathematical operations like cosine similarity
- scikit-learn – for predictive models and clustering (e.g., Random Forest, Gradient Boosting, K-Means)
- pandas – for data manipulation and preprocessing

Installation was done using:

```
pip install flask flask-login pymongo bcrypt requests numpy scikit-learn pandas
```